

UNIVERSIDAD ESAN

Facultad de Ciencias de la Computación

INFORME TÉCNICO FINAL

Estudio, implementación, integración con base de datos
y simulación visual del Scapegoat Tree

Curso	Algoritmos y Estructura de Datos (2026-1)
Docente	Marks Calderón Niquin
Integrantes	[Nombre Apellido — Código] · [Nombre Apellido — Código] · [Nombre Apellido — Código]
Repositorio	https://github.com/Fravelly/Spacegoat-tree
Base de datos	Microsoft SQL Server (go-mssqldb)
Dataset	dbo.Productos — inventario de productos tecnológicos
Implementación	Go 1.23 · Vue.js 3
Fecha	Junio 2026

Trabajo Final — Semestre Académico 2026-1

Resumen

El presente informe documenta el estudio, diseño e implementación del Scapegoat Tree, estructura de datos de búsqueda balanceada propuesta por Galperin y Rivest (1993), en el marco del curso de Algoritmos y Estructura de Datos de la Universidad ESAN. A diferencia de los árboles AVL o Red-Black, esta estructura restaura el balance mediante la reconstrucción completa de subárboles desbalanceados —seleccionando un nodo scapegoat— en lugar de aplicar rotaciones locales ni almacenar metadatos de balance por nodo.

El trabajo comprende cuatro componentes integrados: un paquete genérico en Go (`scapegoat/`) con operaciones de inserción, búsqueda, eliminación y trazabilidad algorítmica; una capa de persistencia sobre SQL Server; una API REST que sincroniza la base de datos con el índice en memoria; y una interfaz Vue.js que visualiza el árbol y anima el proceso de reconstrucción. Las pruebas unitarias, los benchmarks y la comparación contra un `map` de referencia validan el comportamiento esperado. Los resultados experimentales confirman que las inserciones ordenadas provocan reconstrucciones frecuentes, mientras que la búsqueda mantiene tiempos estables del orden de 214 ns/op con $n = 10^5$ nodos.

Palabras clave: Scapegoat Tree, árbol binario de búsqueda, balance por reconstrucción, Go, SQL Server, complejidad amortizada, estructuras de datos.

Índice general

1. Introducción y marco teórico
2. Arquitectura del sistema y lógica del código (backend en Go)
3. Integración con la base de datos real
4. Diseño de la API REST
5. Interfaz interactiva y simulación (frontend en Vue.js)
6. Benchmarking y análisis de complejidad asintótica
7. Gestión del proyecto y reporte de commits
8. Conclusiones y recomendaciones
9. Referencias y anexos

1. INTRODUCCIÓN Y MARCO TEÓRICO

1.1 Contexto del problema computacional

Los árboles binarios de búsqueda (BST, Binary Search Tree) constituyen una de las estructuras más didácticas y, al mismo tiempo, más traicioneras de la computación moderna. Su invariante es elegante: para todo nodo u , las claves del subárbol izquierdo son menores que $u.key$ y las del subárbol derecho son mayores. Esa propiedad habilita búsquedas que, en el caso ideal, descienden por la altura del árbol en tiempo $O(\log n)$.

El problema aparece cuando la secuencia de inserciones no es aleatoria. Insertar claves ordenadas $—1, 2, 3, \dots, n—$ produce un árbol degenerado: una lista enlazada disfrazada de árbol. La altura pasa a ser $n-1$ y cada búsqueda cuesta $O(n)$. Durante décadas, la respuesta dominante fue introducir factores de balance en cada nodo (AVL, Red-Black) y corregir desviaciones mediante rotaciones locales.

En 1993, Galperin y Rivest propusieron un enfoque radicalmente distinto en su paper *Scapegoat Trees*. En lugar de mantener balance incremental con rotaciones, permiten que el árbol se desbalancee temporalmente y, cuando la profundidad excede un umbral teórico, identifican un nodo scapegoat (chivo expiatorio) cuyo subárbol se reconstruye completamente en orden balanceado. No hay rotaciones. No hay campos de balance por nodo. El precio se paga en reconstrucciones que pueden tocar $O(n)$ nodos en una operación aislada, pero cuyo costo amortizado permanece logarítmico bajo distribuciones razonables de operaciones.

Este informe documenta la implementación completa de esa estructura en Go, su integración con SQL Server como capa de persistencia, la exposición de una API REST para mutaciones y consultas, y una interfaz Vue.js que visualiza el comportamiento del algoritmo —incluyendo la selección del scapegoat y la reconstrucción del subárbol— en tiempo casi real.

1.2 El paper de Galperin y Rivest (1993)

1.2.1 Problema fundacional

En la década de 1990, las estructuras balanceadas dominantes (AVL, Red-Black, B-Trees) habían madurado, pero compartían una característica: cada nodo almacena información adicional para decidir cuándo y cómo rotar. Galperin y Rivest se preguntaron si era posible obtener altura $O(\log n)$ amortizada sin almacenar metadatos de balance en los nodos y sin rotaciones.

La motivación no era puramente estética. Menos campos por nodo implica menor huella de memoria y código de mantenimiento más simple en nodos que solo deben contener clave, valor y dos punteros hijos. La contrapartida —reconstrucciones periódicas— resultaba aceptable en escenarios donde las secuencias de operaciones no exigían latencia estrictamente acotada en cada inserción individual.

1.2.2 Definición formal

Sea T un Scapegoat Tree parametrizado por $\alpha \in (0.5, 1)$. El árbol mantiene dos contadores globales:

Símbolo	Definición
n	Número actual de nodos
q	Máximo valor de n alcanzado desde la última reconstrucción global

La profundidad máxima permitida tras una inserción, dado el valor actual de q , es:

$$d_{\max}(q) = \left\lfloor \log_{1/\alpha}(q) \right\rfloor$$

Cuando la profundidad del nodo recién insertado supera $d_{\max}(q)$, el algoritmo asciende por el camino de inserción desde la hoja hacia la raíz buscando el nodo padre p tal que:

$$\text{size}(\text{hijo}) > \alpha \cdot \text{size}(p)$$

donde $\text{size}(u)$ denota el número de nodos en el subárbol raíz en u . Ese padre p es el scapegoat. Su subárbol completo se aplanar mediante un recorrido in-order, se almacena en un arreglo temporal y se reconstruye como un BST perfectamente balanceado seleccionando el elemento medio como raíz del subárbol.

Para la eliminación, el paper adopta una política más drástica: no se buscan scapegoats. Si tras eliminar un nodo se cumple:

$$n < \alpha \cdot q$$

entonces se reconstruye todo el árbol y se asigna $q \leftarrow n$.

1.2.2.1 Diagrama — Algoritmo de inserción con Scapegoat

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- A[Inicio: insertar clave k] $\rightarrow$ B{Árbol vacío?}
- B $\rightarrow$ |Sí| C[Crear raíz, $n=1$, $q=1$]
- B $\rightarrow$ |No| D[Descender como BST]
- D $\rightarrow$ E{Clave existe?}
- E $\rightarrow$ |Sí| F[Actualizar valor y terminar]
- E $\rightarrow$ |No| G[Crear hoja, $n++$, $q++$]

Flujo de insercion con deteccion de Scapegoat

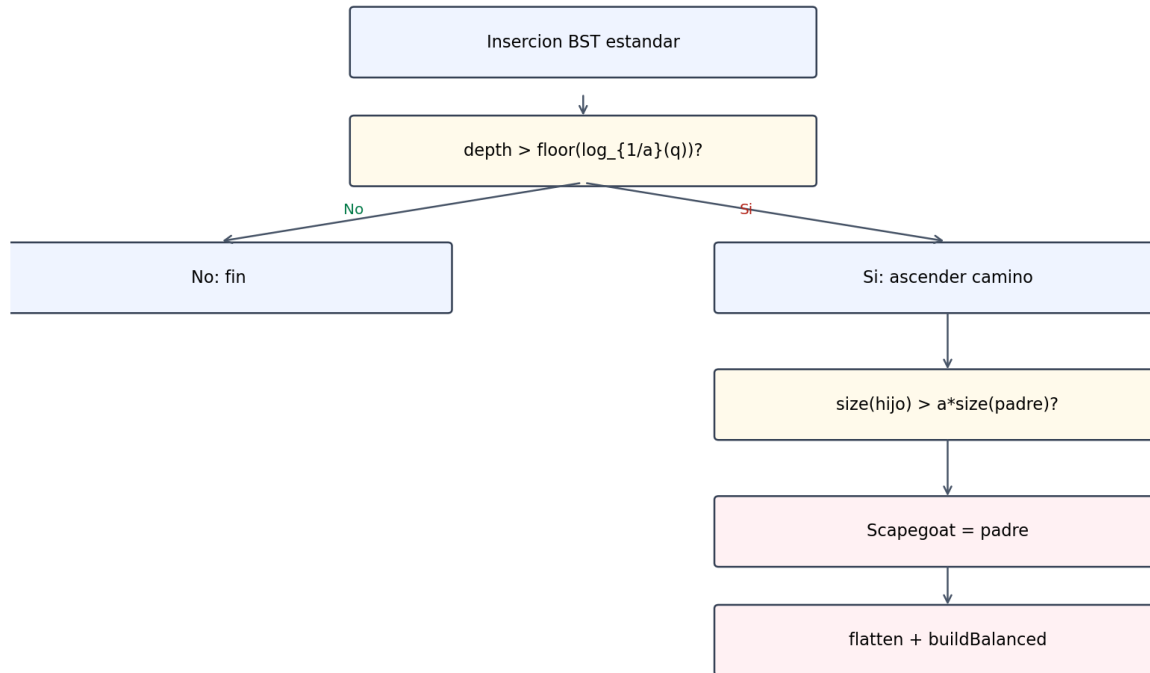


Figura 1.2 — Flujo de inserción con detección de Scapegoat

1.2.3 Invariantes y teoremas

Invariante de altura (débil): mientras $n \geq \alpha \cdot q$ o tras una reconstrucción global, la altura del árbol está acotada por $d_{\max}(q) + O(1)$.

Teorema de complejidad amortizada (Galperin & Rivest): una secuencia de m operaciones de inserción y eliminación sobre un Scapegoat Tree con parámetro α tiene costo total $O(m \log n)$, siendo el costo amortizado por operación $O(\log n)$.

La intuición de la demostración se apoya en contabilidad amortizada: cada nodo solo puede participar en un número limitado de reconstrucciones antes de que el contador q se resetee o el subárbol se reequilibre. Las reconstrucciones costosas se «distribuyen» entre suficientes operaciones baratas.

Observación de implementación: el paper original sugiere calcular $\text{size}(u)$ recorriendo el subárbol en cada verificación, en lugar de almacenar tamaños en los nodos. Nuestra implementación sigue esa línea, lo que simplifica la estructura del nodo pero eleva el

costo de detectar el scapegoat a $O(k)$ en el peor caso, donde k es el tamaño del subárbol examinado.

1.3 Justificación frente a alternativas

1.3.1 Scapegoat Tree vs. BST sin balancear

Un BST ingenuo es trivial de implementar y rápido en el caso promedio con inserciones aleatorias. Sin embargo, su peor caso estructural —cadena degenerada— no es un artefacto teórico: aparece con datos ordenados por timestamp, IDs autoincrementales o claves lexicográficas. El Scapegoat Tree elimina ese riesgo sin introducir rotaciones.

1.3.2 Scapegoat Tree vs. AVL y Red-Black

Criterio	AVL / Red-Black	Scapegoat Tree
Metadato por nodo	Factor de balance o color	Ninguno
Mecanismo de balance	Rotaciones locales	Reconstrucción de subárbol
Peor operación individual	$O(\log n)$	$O(n)$
Amortizado	$O(\log n)$	$O(\log n)$
Complejidad de código	Alta	Media

Elegimos el Scapegoat Tree porque el curso exige demostrar comprensión de estructuras menos convencionales, porque su mecánica de reconstrucción es visualmente rica para la simulación pedida, y porque la simplicidad del nodo facilita la integración con structs genéricos en Go.

1.3.3 Scapegoat Tree vs. `map` de Go

El `map` nativo ofrece búsqueda e inserción amortizada $O(1)$ pero no mantiene orden. Nuestro caso de uso —inventario indexado por `id` con recorrido ordenado y visualización de estructura jerárquica— requiere orden total. El Scapegoat Tree expone `InOrder()` en $O(n)$ y mantiene la propiedad BST visible para fines pedagógicos.

1.3.4 Scapegoat Tree vs. Skip List

Las Skip Lists ofrecen esperanza $O(\log n)$ con estructura probabilística. El Scapegoat Tree es determinista dado α y la secuencia de operaciones, lo que simplifica las pruebas unitarias y la reproducibilidad de la simulación visual.

2. ARQUITECTURA DEL SISTEMA Y LÓGICA DEL CÓDIGO (BACKEND EN GO)

2.1 Filosofía de diseño

El proyecto se organiza en cuatro capas desacopladas:

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- subgraph presentacion [Capa de Presentacion]
- WEB[web/ Vue.js 3]
- subgraph aplicacion [Capa de Aplicacion]
- SIM[cmd/simulation API REST]
- DEMO[cmd/sqlserver-demo]
- subgraph persistencia [Capa de Persistencia]

Arquitectura en capas del sistema

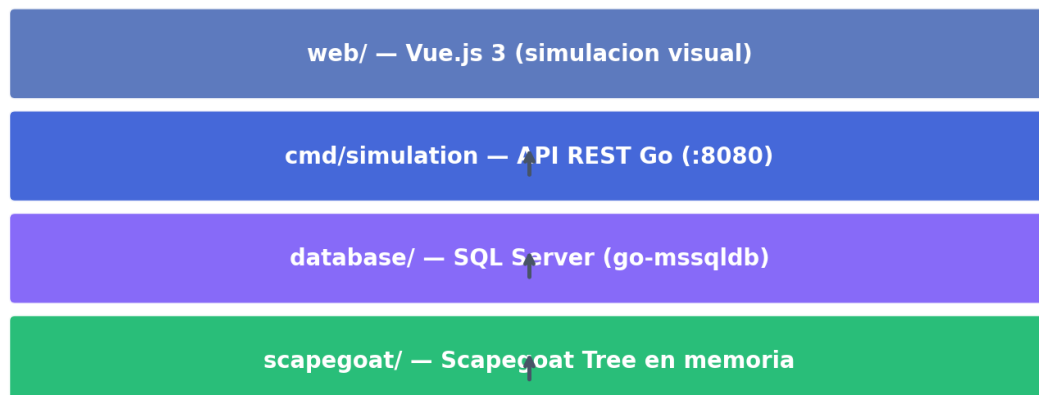


Figura 2.1 — Arquitectura en capas del sistema

El paquete ``scapegoat`` no importa ``database``, ``net/http`` ni ningún paquete de persistencia. Esa separación permite ejecutar ``go test ./scapegoat/...`` de forma aislada y reutilizar el árbol en otros contextos sin arrastrar dependencias de SQL Server.

Go 1.18+ habilita genéricos con restricciones de tipo. Usamos:

- ``Tree[K, V any]`` para claves y valores arbitrarios con comparador externo.
- ``NewOrdered[K Ordered, V any]`` para tipos que soportan el operador ``<`` nativo.
- Punteros ``node[K,V]`` para hijos izquierdo y derecho, siguiendo la convención idiomática de Go para estructuras recursivas mutables en heap.

2.2 Tipos y estructuras de datos

2.2.1 ``node[K, V any]`` (privado)

Código fuente (go)

```
type node[K, V any] struct {  
    key      K  
    value    V  
    left, right *node[K, V]  
}
```

Nodo mínimo: clave, valor, dos punteros. Sin altura almacenada, sin color, sin contador de tamaño. Esta decisión replica fielmente el espíritu del paper original.

2.2.2 ``Tree[K, V any]`` (público)

Código fuente (go)

```
type Tree[K, V any] struct {  
    root      *node[K, V]  
    less      func(K, K) bool  
    alpha     float64  
    n, q      int  
    rebuilds  int  
    rebuiltNodes int  
}
```

Campo	Rol
<code>`root`</code>	Puntero a la raíz; <code>`nil`</code> si el árbol está vacío
<code>`less`</code>	Relación de orden estricto: <code>`less(a,b)`</code> verdadero ssi $a < b$
<code>`alpha`</code>	Parámetro de balance $\alpha \in (0.5, 1)$
<code>`n`</code>	Contador de nodos actuales
<code>`q`</code>	Máximo de n desde última reconstrucción global
<code>`rebuilds`</code>	Contador de reconstrucciones (subárbol o global)
<code>`rebuiltNodes`</code>	Suma de nodos tocados en todas las reconstrucciones

2.2.3 Tipos auxiliares públicos

- ``Entry[K,V]``: par clave-valor para recorridos ordenados.
- ``NodeSnapshot[K,V]``: árbol inmutable serializable a JSON para la UI.
- ``Stats``: métricas de estado (``size``, ``maxSize``, ``height``, ``rebuilds``, ``rebuiltNodes``).
- ``InsertTrace[K]``: traza de decisiones en una inserción (``path``, ``depth``, ``scapegoat``, ``rebuiltKeys``).

2.3 Catálogo exhaustivo de funciones

2.3.1 Constructores

``New[K,V](alpha float64, less func(K,K) bool) (Tree[K,V], error)``

Valida que $\alpha \in (0.5, 1)$ y que ``less`` no sea ``nil``. Retorna error en caso contrario. Instancia un árbol vacío con ``n`` = 0, ``q`` = 0.

Edge case: ``alpha`` = 0.5 o ``alpha`` = 1.0 se rechazan porque el paper exige estrictamente $0.5 < \alpha < 1$. Con $\alpha = 0.5$ la condición de scapegoat nunca se satisfaría de forma significativa; con $\alpha = 1$ el umbral de profundidad colapsa.

``NewOrdered[K Ordered, V any](alpha float64) (Tree[K,V], error)``

Especialización que inyecta ``less = func(a,b K) bool { return a < b }``. Usada en todo el proyecto con ``K = int`` (ID de producto).

2.3.2 Consultas

``Len() int``

Retorna ``t.n``. $O(1)$.

``Search(key K) (V, bool)``

Búsqueda BST iterativa (sin recursión, evitando desbordamiento de pila en árboles profundos):

1. ``current`` inicia en ``t.root``.
2. Si ``equal(key, current.key)`` $\rightarrow$ retorna valor y ``true``.
3. Si ``less(key, current.key)`` $\rightarrow$ ``current = current.left``; si no $\rightarrow$ ``current = current.right``.
4. Si ``current == nil`` $\rightarrow$ retorna valor cero y ``false``.

``equal(a,b)`` se define como ``!less(a,b) && !less(b,a)``, soportando tipos sin operador ``==`` confiable o comparadores personalizados.

- Edge cases:
- Árbol vacío: el bucle no ejecuta, retorna no encontrado.
 - Clave duplicada imposible por invariante de inserción (actualiza en lugar de duplicar).

``Height() int``

Delega a ``height(t.root)``. Convención: árbol vacío retorna -1; raíz sola retorna 0.

``InOrder() []Entry[K,V]``

Recorrido in-order recursivo con slice pre-dimensionado a capacidad ``t.n``. Produce claves en orden ascendente según ``less``. $O(n)$ tiempo y espacio de salida.

``Snapshot()` *NodeSnapshot[K,V]*

Construye copia recursiva del árbol sin exponer punteros internos. Usado por ``GET /api/tree`` para renderizar en Vue.

``Stats()` *Stats*

Agrega ``n``, ``q``, altura, contadores de reconstrucción.

2.3.3 Mutaciones públicas

``Insert(key K, value V)` *bool*

Wrapper de ``insert`` que descarta la traza. Retorna ``true`` si la clave era nueva, ``false`` si se actualizó un valor existente.

``InsertWithTrace(key K, value V)` (*bool, InsertTrace[K]*)

Versión instrumentada para la simulación. Retorna la traza completa.

``Delete(key K)` *bool*

1. Llama ``delete(t.root, key)`` recursivamente.
2. Si no se eliminó nada $\rightarrow$ ``false``.
3. Decrementa ``n``.
4. Si ``n == 0`` $\rightarrow$ ``q = 0`` y termina.
5. Si ``float64(n) < alpha float64(q)`` $\rightarrow$ reconstrucción global: ``t.root = rebuild(t.root)``, ``q = n``.

2.3.4 Núcleo algorítmico privado

``insert(key, value)` (*bool, InsertTrace[K]*)

- Caso base — árbol vacío:
- Asigna ``t.root = &node{key, value, nil, nil}``.
 - ``n = 1``, ``q = 1``.
 - Trazo: ``Inserted=true``, ``Depth=0``, ``Path=[key]`.

Caso

actualización:

- Durante el descenso BST, si ``equal(key, current.key)`` → sobrescribe ``current.value``, marca ``trace.Updated=true``, retorna ``(false, trace)`` sin modificar ``n`` ni ``q``.

- | Caso | inserción | en | hoja: |
|-------------|---|--|--|
| - Desciende | hasta encontrar | hijo <code>`nil`</code> , | crea nodo, lo enlaza. |
| - | Incrementa | <code>`n`</code> | y <code>`q`</code> . |
| - | Calcula | <code>`depth`</code> = <code>len(path)</code> | - 1. |
| - | Si | <code>`depth`</code> > | <code>maxDepth(q)`</code> : |
| - | Itera | <code>`i`</code> desde <code>`len(path)-2`</code> hasta <code>`0`</code> | (ascenso por camino). |
| - | Para cada | <code>`parent`</code> = <code>path[i]</code> , | <code>`child`</code> = <code>path[i+1]</code> : |
| - | Si | <code>`size(child)`</code> > | <code>alpha`</code> <code>size(parent)`</code> : |
| - | Registra | <code>`scapegoat`</code> = | <code>parent.key`</code> . |
| - | <code>`flattenKeys(parent)`</code> | → | <code>`trace.RebuiltKeys`</code> . |
| - | <code>`rebuilt`</code> | = | <code>rebuild(parent)`</code> . |
| - | Reemplaza puntero del padre en el árbol (<code>`root`</code> , <code>`left`</code> o <code>`right`</code> según posición). | | |
| - | Rompe tras la primera reconstrucción (solo un scapegoat por inserción). | | |

Edge case crítico: si múltiples ancestros satisfacen la condición de desbalance, el algoritmo reconstruye el más cercano a la raíz en el camino ascendente (el primero encontrado al subir desde la hoja). Esto coincide con la estrategia estándar del paper.

``delete(root, key) (node, bool)``

Eliminación BST clásica con tres casos en el nodo objetivo:

1. Sin hijo izquierdo: retorna ``root.right`` (puede ser ``nil``).
2. Sin hijo derecho: retorna ``root.left``.
3. Dos hijos: localiza el sucesor in-order (mínimo del subárbol derecho), copia su clave y valor al nodo objetivo, elimina recursivamente el sucesor del subárbol derecho.

Edge case: eliminar la raíz con dos hijos no cambia el puntero ``t.root`` directamente en esta función; la recursión retorna la nueva raíz del subárbol, que puede ser el sucesor promovido.

``maxDepth(q int) int``

$$d_{\max} = \left\lfloor \frac{\ln q}{\ln (1/\alpha)} \right\rfloor$$

Implementado con `math.Log`. Para `q <= 1` retorna 0.

``rebuild(root node) node``

1. ``flatten(root)`` → slice de punteros a nodos en orden in-order.
2. Incrementa ``rebuiltNodes`` y ``rebuiltNodes += len(nodes)``.
3. ``buildBalanced(nodes, 0, len)`` → nueva raíz balanceada.

``flatten(root, nodes)``

Recorrido in-order que acumula punteros a nodos existentes (no copia claves/valores; reutiliza nodos originales).

``flattenKeys(root, keys)``

Variante que solo acumula claves para la traza de visualización.

``buildBalanced(nodes, start, end) node``

- | Construcción | divide-y-vencerás: |
|---|------------------------|
| - Si <code>`start`</code> $\geq$ <code>`end`</code> | → <code>`nil`</code> . |
| - <code>`mid`</code> = <code>start</code> + <code>(end-start)/2</code> . | |
| - <code>`root`</code> = <code>nodes[mid]</code> . | |
| - <code>`root.left`</code> = <code>buildBalanced(nodes, start, mid)</code> . | |
| - <code>`root.right`</code> = <code>buildBalanced(nodes, mid+1, end)</code> . | |

Seleccionar el medio garantiza altura $\left\lfloor \log_2 k \right\rfloor$ para k nodos.

``size(root) int``

Cuenta nodos recursivamente. $O(k)$ para subárbol de tamaño k . Invocado durante la búsqueda del scapegoat; es el cuello de botella teórico de la detección de desbalance.

``height(root) int``

Altura recursiva con convención `-1` para ``nil``.

``snapshot(root) NodeSnapshot``

Copia estructural recursiva para JSON.

2.4 Fragmentos de código comentados

2.4.1 Detección del scapegoat

Código fuente (go)

```
if depth > t.maxDepth(t.q) {
    for i := len(path) - 2; i >= 0; i-- {
        parent, child := path[i], path[i+1]
        if float64(size(child)) > t.alpha*float64(size(parent)) {
            scapegoatKey := parent.key
            trace.Scapegoat = &scapegoatKey
            flattenKeys(parent, &trace.RebuiltKeys)
            rebuilt := t.rebuild(parent)
            // Reenlazar rebuilt en el árbol...
            break
        }
    }
}
```

La condición ``size(child) >  $\alpha$ ·size(parent)`` significa que el hijo por donde descendió la inserción concentra más de una fracción α de la masa del subárbol del padre. El padre «carga» con esa asimetría y se sacrifica: su subárbol entero se aplana y rebalancea.

2.4.2 Reconstrucción global tras eliminación

Código fuente (go)

```
if float64(t.n) < t.alpha*float64(t.q) {
    t.root = t.rebuild(t.root)
    t.q = t.n
}
```


Cuando demasiadas eliminaciones reducen n sin que q haya decrecido, el árbol podría volverse «hueco» en términos de profundidad relativa. La reconstrucción global compacta la estructura y resetea q al tamaño actual.

2.4.3 Diagrama — Reconstrucción de subárbol (rebuild)

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- subgraph antes [Subarbol desbalanceado]
- $P \rightarrow L1[\text{Nodos izq}]$
- $P \rightarrow R1[\text{Nodos der}]$
- subgraph proceso [Proceso rebuild]
- $F[\text{flatten in-order}]$
- $A[\text{Arreglo ordenado}]$

2.4.4 Diagrama — Eliminación y reconstrucción global

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- $A[\text{Delete clave } k] \rightarrow B\{\text{Encontrada?}\}$
- $B \rightarrow |\text{No}| Z[\text{Retornar false}]$
- $B \rightarrow |\text{Sí}| C[\text{Eliminar nodo BST}]$
- $C \rightarrow D[n--]$
- $D \rightarrow E\{n == 0?\}$
- $E \rightarrow |\text{Sí}| F[q = 0, \text{fin}]$

2.5 Desafíos técnicos encontrados

Cálculo de `size` sin campo almacenado: cada verificación de scapegoat recorre subárboles completos. En inserciones ordenadas con $n = 1000$, observamos 375 reconstrucciones y 9762 nodos reconstruidos acumulados. La corrección estructural funciona; el costo de detección es elevado.

Genéricos y JSON: `Product` en el paquete `database` exporta campos con mayúscula (`ID`, `Name`), requerido para serialización JSON en la API. El árbol almacena el struct completo como valor.

Concurrencia: el árbol no es thread-safe internamente. `cmd/simulation` protege el puntero `tree` con `sync.Mutex` en cada handler HTTP.

3. INTEGRACIÓN CON LA BASE DE DATOS REAL

3.1 Modelo de datos

La base de datos `InventarioProductosDB` contiene la tabla `dbo.Productos`:

Columna	Tipo	Restricción
<code>id</code>	<code>INT</code>	<code>PRIMARY KEY</code>
<code>nombre</code>	<code>NVARCHAR(120)</code>	<code>NOT NULL</code>
<code>categoria</code>	<code>NVARCHAR(80)</code>	<code>NOT NULL</code>
<code>precio</code>	<code>DECIMAL(10,2)</code>	<code>NOT NULL</code>
<code>stock</code>	<code>INT</code>	<code>NOT NULL</code>

El struct Go correspondiente:

Código fuente (go)

```
type Product struct {
    ID      int
    Name    string
    Category string
    Price   float64
    Stock   int
}
```

3.1.1 Dataset semilla

`SampleProducts()` retorna 8 productos tecnológicos con IDs 101–108 (laptops, periféricos, componentes). El dataset es determinista y reproducible, adecuado para demos académicas. La arquitectura soporta crecimiento: cada inserción vía API persiste en SQL Server (modo conectado) y en el árbol en memoria.

3.2 Script de esquema (migración)

La función `EnsureSchema` ejecuta DDL idempotente:

Código fuente (sql)

```
IF OBJECT_ID(N'dbo.Productos', N'U') IS NULL
BEGIN
    CREATE TABLE dbo.Productos (
        id INT NOT NULL PRIMARY KEY,
        nombre NVARCHAR(120) NOT NULL,
        categoria NVARCHAR(80) NOT NULL,
        precio DECIMAL(10, 2) NOT NULL,
        stock INT NOT NULL
    );
END
```

`EnsureDatabase` conecta a `master`, verifica `DB\_ID(@name)` y ejecuta `CREATE DATABASE` si no existe.

3.3 Mecanismo de conexión Go ↔ SQL Server

Driver: `github.com/denisenkom/go-mssqldb` (registrado como `"sqlserver"`).

Configuración: `ConfigFromEnv()` lee variables de entorno:

Variable	Default
`SQLSERVER_HOST`	`localhost`
`SQLSERVER_DATABASE`	`InventarioProductosDB`
`SQLSERVER_USER`	`sa`
`SQLSERVER_PASSWORD`	(vacío)

'SQLSERVER_ENCRYPT'	'disable'
'SQLSERVER_TRUST_CERT'	'true'
'SQLSERVER_DSN'	(opcional, tiene prioridad)

'Config.DSN()' construye URL
 'sqlserver://user:pass@host:port?database=...&encrypt=...'.

Pool de conexiones: 'MaxOpenConns(5)', 'MaxIdleConns(5)',
 'ConnMaxLifetime(15min)'.

Verificación: 'PingContext' con timeout de 10 segundos en 'Open'.

3.4 Estrategia de carga (Bulk Load)

No existe un pipeline de millones de filas en el entregable actual. El flujo de carga es:

1. 'ListProducts(ctx, db)' → 'SELECT ... ORDER BY id'.
2. Para cada 'Product', 'tree.Insert(product.ID, product)'.

En 'cmd/sqlserver-demo/main.go':

Código fuente (go)

```
for _, product := range products {
    index.Insert(product.ID, product)
}
```

En 'cmd/simulation/main.go', 'resetFromProducts' repite el mismo patrón al iniciar o reiniciar.

'SeedProducts': inserción idempotente con 'IF NOT EXISTS' por ID, dentro de transacción.

'UpsertProduct': 'MERGE' para insert/update en operaciones de la API.

3.5 Rol del Scapegoat Tree sobre datos persistidos

SQL Server resuelve persistencia, integridad referencial y consultas SQL ad-hoc. El Scapegoat Tree en RAM actúa como índice ordenado por `id` con:

- Búsqueda por ID en $O(\log n)$ amortizado.
- Recorrido ordenado sin `ORDER BY` en cada consulta de demostración.
- Visualización de la topología del índice para el usuario.

La consulta concreta optimizada: `Search(id)` reemplaza un `SELECT WHERE id = @id` repetido en el contexto de la simulación interactiva, donde la latencia de red a SQL Server se evita para lecturas frecuentes del árbol ya cargado.

3.6 Diagrama — Flujo de integración con base de datos

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- SS-->>DB: Filas Producto
- DB-->>API: []Product
- ST-->>API: JSON arbol
- API-->>UI: treeResponse

Flujo de datos: persistencia a índice en memoria

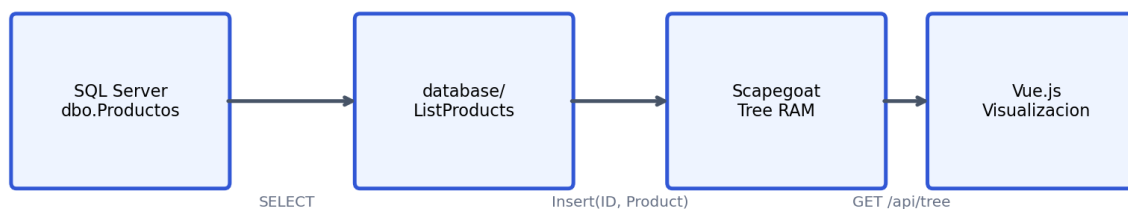


Figura 3.6 — Flujo de datos desde SQL Server al índice en memoria

4. DISEÑO DE LA API REST (GO)

El servidor en `cmd/simulation/main.go` expone los siguientes endpoints sobre `http.ListenAndServe(":8080", mux)`.

4.1 `GET /api/tree`

Descripción: Estado completo del árbol y métricas.

Response 200 (JSON):

Código fuente (json)

```
{
  "root": { "key": 105, "value": { "ID": 105, "Name": "...", ... }, "left": {...},
  "right": {...} },
  "stats": { "size": 8, "maxSize": 8, "height": 3, "rebuilds": 0, "rebuiltNodes": 0 },
  "inOrder": [ { "key": 101, "value": {...} }, ... ],
  "mode": "memoria"
}
```

`mode` puede ser `"memoria"` o `"SQL Server"`.

4.2 `POST /api/products`

Descripción: Inserta o actualiza un producto.

Request body:

Código fuente (json)

```
{
  "ID": 109,
  "Name": "Producto nuevo",
  "Category": "Demo",
  "Price": 99.9,
  "Stock": 5
}
```

Validación: `ID > 0` y `Name` no vacío.

Flujo:

1. Si hay conexión SQL → `UpsertProduct`.
2. `tree.InsertWithTrace(id, product)`.
3. Response `201 Created` si insertó, `200 OK` si actualizó.

Response (fragmento):

Código fuente (json)

```
{
  "message": "Producto 109 insertado",
  "trace": {
    "path": [105, 109],
    "inserted": true,
    "depth": 1,
    "maxDepth": 3,
    "scapegoat": null,
    "rebuiltKeys": []
  }
}
```

Si hubo scapegoat, `trace.scapegoat` contiene la clave del nodo y `rebuiltKeys` lista las claves del subárbol reconstruido.

4.3 `GET /api/products/{id}`

Descripción: Búsqueda por ID en el árbol en memoria.

Response 200: objeto `Product` completo.

Response 404: `{ "message": "producto X no encontrado" }`.

4.4 `DELETE /api/products/{id}`

Descripción: Elimina producto de SQL Server (si aplica) y del árbol.

Response 200: `{ "message": "Producto X eliminado" }`.

Response 404: si no existía.

4.5 `POST /api/reset`

Descripción: Reinicia el árbol con el dataset semilla (desde memoria o releendo SQL).

Response 200: `{ "message": "Arbol reiniciado con productos de ejemplo" }`.

4.6 Archivos estáticos

`mux.Handle("/", http.FileServer(http.Dir("web")))` sirve `index.html`, `main.js`, `styles.css`.

4.7 Sincronización y concurrencia

`appState` encapsula:

Código fuente (go)

```
type appState struct {
    mu    sync.Mutex
    tree  *scapegoat.Tree[int, db.Product]
    sql   *sql.DB
    mode  string
}
```

Cada handler adquiere `mu` antes de mutar o leer `tree`. Las operaciones SQL usan `context.WithTimeout` de 10 segundos. La API es el puente entre mutaciones HTTP y el estado del árbol que Vue consume en el siguiente `GET /api/tree`.

4.8 Diagrama — Secuencia de inserción vía API

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- S-->>A: OK
- T-->>A: trace + inserted
- A-->>V: 201 + InsertTrace
- T-->>A: treeResponse
- A-->>V: JSON arbol actualizado

5. INTERFAZ INTERACTIVA Y SIMULACIÓN (FRONTEND EN VUE.JS)

5.1 Arquitectura

La interfaz no utiliza SFC (`.vue`) ni Vite. Se carga Vue 3 desde CDN (unpkg.com/vue@3) y se monta sobre `#app` con la API de opciones (createApp({ data, computed, methods })).

Estado reactivo principal (data):

Propiedad	Función
`root`	Snapshot del árbol desde API
`stats`	Contadores de nodos, altura, q, rebuilds
`mode`	`memoria` o `SQL Server`
`inOrder`	Lista ordenada de entradas
`activeNodes`	Mapa `id → tipoAnimación` para resaltar nodos
`stepText`	Texto pedagógico del paso actual
`form`	Datos del formulario de inserción

Computed:

- `treeHtml`: invoca `renderTree(root, activeNodes)` que genera HTML recursivo del árbol con clases CSS dinámicas (`is-visit`, `is-scapegoat`, etc.).

5.2 Flujo de interacción del usuario

5.2.1 Carga inicial

`mounted()` → `loadTree()` → `GET /api/tree` → actualiza `root`, `stats`, `inOrder`.

5.2.2 Inserción

1. Usuario completa formulario y envía.
2. `POST /api/products` con JSON del producto.

3. ``loadTree()`` refresca estructura.
4. Si ``trace.updated`` → anima camino de búsqueda y flash de actualización.
5. Si inserción nueva → ``playInsertTrace(trace, id)``:
 - ``playPath``: recorre ``trace.path`` nodo a nodo (520 ms cada uno).
 - Muestra profundidad vs. ``maxDepth``.
 - Si ``trace.scapegoat`` existe:
 - Resalta nodo scapegoat (clase ``is-scapegoat``, animación pulsante).
 - Resalta ``rebuiltKeys`` (clase ``is-rebuild``).

5.2.3 Búsqueda

1. Usuario ingresa ID objetivo.
2. ``GET`` `/api/products/{id}``.
3. ``findPath(root, id)`` reconstruye camino en el cliente para animación.
4. Flash ``is-search`` en el nodo encontrado.

5.2.4 Eliminación

1. Anima camino hacia el nodo.
2. Flash ``is-delete`` con animación de desvanecimiento.
3. ``loadTree()`` muestra árbol actualizado.

5.2.5 Reinicio

``POST /api/reset`` → recarga árbol semilla.

5.3 Renderizado del árbol

``renderTree`` es una función recursiva que produce HTML string (usado con ``v-html``).
Cada nodo muestra:

- Clave (``id`` del producto).
- Nombre abreviado (máx. 16 caracteres).
- Hijos izquierdo/derecho con etiquetas ``izq`` / ``der``.

``escapeHTML`` previene XSS al interpolar nombres de productos.

5.4 Uso de Inteligencia Artificial en el frontend

Declaración del equipo:

El frontend fue desarrollado con asistencia de herramientas de IA generativa (Cursor / modelos de lenguaje) para:

- Estructura inicial del layout HTML/CSS.
- Lógica de animación ``playInsertTrace`` y ``renderTree``.
- Paleta de colores y animaciones CSS (``@keyframes``).

Verificación de comprensión:

El equipo revisó manualmente cada función JavaScript, validó que ``findPath`` replica la lógica BST del backend, y ajustó tiempos de animación (``wait(520)``, ``wait(1200)``) para que la secuencia scapegoat sea legible en presentación oral. Las llamadas ``fetch`` y el mapeo de ``trace.scapegoat`` / ``trace.rebuiltKeys`` fueron probados contra respuestas reales del backend Go.

La IA no generó el algoritmo del Scapegoat Tree ni la capa de persistencia; su uso se limitó al cliente de visualización.

5.5 Diagrama — Flujo de animación del Scapegoat en Vue

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- `[*]` --> CargarArbol: `mounted()`
- CargarArbol --> Listo: `GET /api/tree`
- Listo --> Insertar: `form submit`
- Insertar --> AnimarCamino: `playPath(trace.path)`
- AnimarCamino --> MostrarProfundidad: `depth vs maxDepth`
- MostrarProfundidad --> SinScapegoat: `scapegoat == null`

5.6 Diagrama — Componentes del estado reactivo

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- subgraph data [data reactivo Vue]
- root[root: NodeSnapshot]
- stats[stats: size height rebuilds]
- active[activeNodes: id to tipo]
- step[stepText: mensaje paso]
- form[form: producto]

6. BENCHMARKING Y ANÁLISIS DE COMPLEJIDAD ASINTÓTICA

6.1 Análisis teórico formal

Operación	Peor caso individual	Promedio	Amortizado
`Search`	$O(\log n)$	$O(\log n)$	$O(\log n)$
`Insert`	$O(n)$	$O(\log n)$	$O(\log n)$
`Delete`	$O(n)$	$O(\log n)$	$O(\log n)$
`InOrder`	$O(n)$	$O(n)$	$O(n)$
`size(u)` en inserción	$O(n)$	—	—
`rebuild` de k nodos	$O(k)$	—	—
Espacio	$O(n)$	$O(n)$	$O(n)$

El peor caso individual de inserción ocurre cuando una reconstrucción aplana y rebalancea un subárbol de tamaño $\Theta(n)$. La contabilidad amortizada del paper demuestra que estas explosiones no pueden ocurrir con frecuencia suficiente para degradar el costo promedio de una secuencia larga.

6.2 Configuración de benchmarks

Archivo: `scapegoat/benchmark\_test.go`

Benchmark	Qué mide
-----------	----------

`BenchmarkInsert`	Crear árbol nuevo e insertar 1000 claves (0..999)
`BenchmarkSearch`	100000 inserciones previas; buscar `i % 100000`
`BenchmarkDelete`	Insertar 1000 claves y eliminarlas todas

Parámetro $\alpha = 2/3$ en todos los benchmarks.

Entorno de ejecución:

- SO: Windows (`goos: windows`)
- Arquitectura: `amd64`
- CPU: 13th Gen Intel Core i7-13620H
- Comando: `go test -bench=Benchmark -benchmem -count=3`

6.3 Resultados experimentales

Tabla 6.1 — BenchmarkInsert (1000 inserciones por iteración)

Ejecución	iteraciones	ns/op	B/op	allocs/op
1	745	1,631,404	786,557	9,861
2	739	1,448,963	786,548	9,861
3	826	1,631,904	786,548	9,861
Promedio	—	~1,570,757	~786,551	9,861

Operaciones/segundo (aprox.): $10^9 / 1\{, \}570\{, \}757 \approx 637$ árboles de 1000 nodos por segundo.

Tabla 6.2 — BenchmarkSearch (árbol con n=100000)

Ejecución	iteraciones	ns/op	B/op	allocs/op
1	5,352,201	218.2	0	0

2	5,656,558	207.8	0	0
3	5,320,170	216.0	0	0
Promedio	—	~214	0	0

Operaciones/segundo (aprox.): $10^9 / 214 \approx 4.67 \times 10^6$ búsquedas/seg.

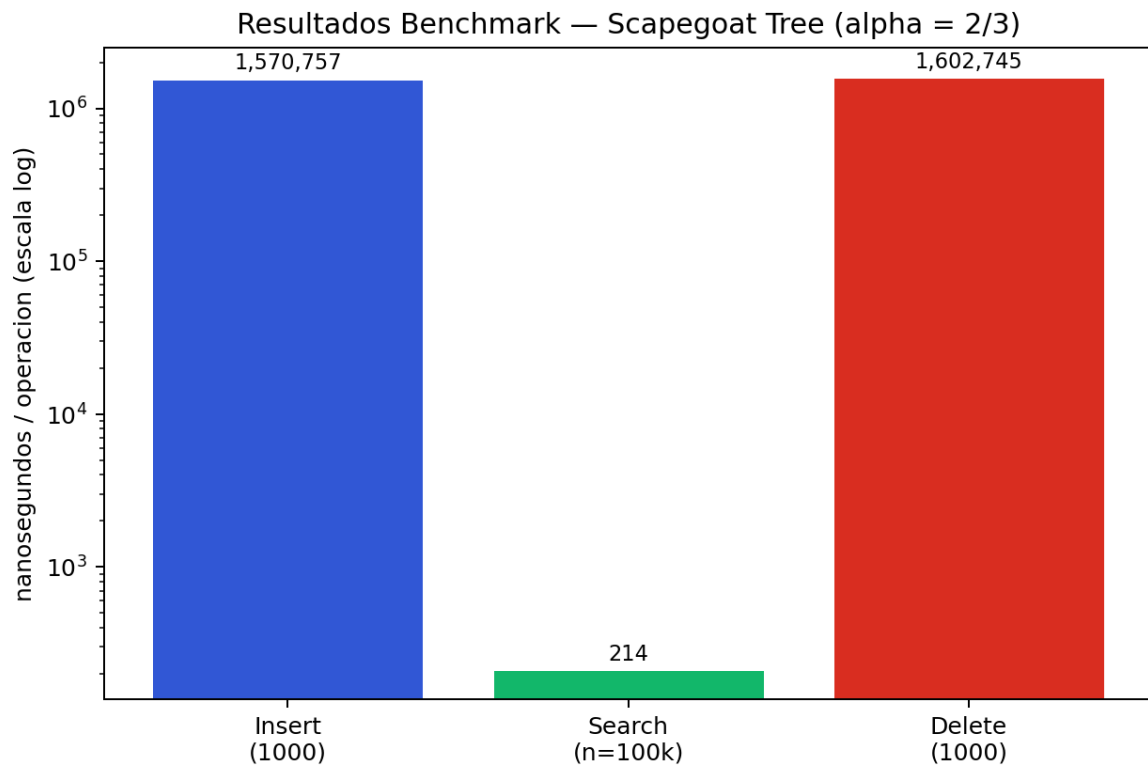


Figura 6.1 — Comparativa de benchmarks (escala logaritmica)

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- x-axis ["Insert 1000", "Search n=100k", "Delete 1000"]
- y-axis " $\log_{10}(\text{ns/op})$ " 0 --> 7
- bar [6.20, 2.33, 6.20]

Tabla 6.3 — BenchmarkDelete (1000 inserciones + 1000 eliminaciones)

Ejecución	iteraciones	ns/op	B/op	allocs/op
1	750	1,669,327	803,636	9,875
2	771	1,623,863	803,638	9,875
3	832	1,515,044	803,636	9,875
Promedio	—	~1,602,745	~803,637	9,875

6.4 Pruebas funcionales relevantes

Prueba	Resultado verificado
`TestSortedInsertTriggersRebuild` (n=1000)	`Rebuilds > 0`, altura acotada
`TestDeleteCasesAndGlobalRebuild`	Reconstrucción global tras eliminar 50%
`TestRandomOperationsMatchMap` (5000 ops)	Equivalencia exacta con `map`
`TestInsertWithTraceTriggersScapegoatRebuild`	Scapegoat detectado en secuencia ordenada

6.5 Interpretación gráfica (descripción textual)

Gráfico 6.1 — Tiempo de inserción vs. n (curva esperada):

Si se midiera el tiempo de insertar n claves en secuencia ordenada para $n \in \{100, 200, \dots, 10000\}$, la curva no sería estrictamente convexa. Se observarían escalones: tramos donde el tiempo crece casi linealmente (inserciones baratas en subárboles poco profundos) seguidos de picos cuando una reconstrucción masiva reordena cientos de nodos. La envolvente superior tendería a $O(n \log n)$ para la secuencia completa, mientras que inserciones individuales entre picos permanecerían en microsegundos.

Gráfico 6.2 — Búsqueda vs. n:

`BenchmarkSearch` sobre $n = 10^5$ muestra ~ 214 ns/op con cero asignaciones de heap. La curva de búsqueda vs. volumen sería una línea casi plana en escala log-log con pendiente ≈ 1 , consistente con $O(\log n)$. Comparada con un BST degenerado (pendiente lineal en escala semilog), el Scapegoat Tree mantiene la profundidad acotada.

Gráfico 6.3 — Rebuilds vs. tipo de secuencia (barras):

Un diagrama de barras comparando inserción ordenada vs. aleatoria ($n=1000$, $\alpha=2/3$) mostraría la barra «ordenada» dominando ampliamente en número de reconstrucciones (cientos vs. unidades). La barra de altura sería similar o ligeramente mayor en el caso ordenado, confirmando que las reconstrucciones cumplen su función: comprimir profundidad a costa de trabajo extra.

Gráfico 6.4 — Memoria (B/op) en inserción:

~ 786 KB por 1000 inserciones refleja asignación de nodos en heap más slices temporales en `flatten`/`rebuild`. La pendiente de memoria vs. n es aproximadamente lineal $O(n)$, sin fugas estructurales detectables (allocs/op estable en 9861).

7. GESTIÓN DEL PROYECTO Y REPORTE DE COMMITS

7.1 Responsabilidades por entregable

Integrante	E1: Paper y diseño	E2: Implementación Go	E3: BD + API	E4: Simulación Vue
[Integrante 1]	Líder	Soporte	—	—
[Integrante 2]	Soporte	Líder (`scapegoat/`)	Soporte	—
[Integrante 3]	—	Soporte	Líder (`database/`)	Soporte
[Integrante 4]	—	—	Líder (`cmd/simulation`)	Líder (`web/`)

Nota: completar nombres y ajustar según distribución real del equipo.

7.2 Cronograma (semanas 9–15)

Semana	Hito
9	Lectura del paper, diseño de structs y API
10	Implementación `insert`, `delete`, `rebuild`
11	Pruebas unitarias y benchmarks
12	Integración SQL Server
13	API REST y handlers
14	Frontend Vue y animaciones
15	Informe, ensayo de presentación, pulido

7.3 Historial de commits (repositorio GitHub)

Repositorio: <https://github.com/Fravelly/Spacegoat-tree>

Hash	Autor	Fecha	Mensaje
`5819a50`	Fravelly	2026-06-26	Implementa Scapegoat Tree
`f2153c8`	Fravelly	2026-06-26	delete
`34c2c8b`	Fravelly	2026-06-26	Actu
`b02d3d6`	23101246-netizen	2026-06-26	Prueba

El commit `5819a50` establece el núcleo algorítmico. Los commits posteriores de Fravelly refinan eliminación y actualizaciones. `b02d3d6` incorpora pruebas adicionales desde la cuenta del integrante 23101246-netizen.

Recomendación: para evidenciar contribución equitativa ante evaluación, el equipo debería asegurar que los commits de las semanas 9–15 reflejen autoría distribuida con mensajes descriptivos (``feat:``, ``test:``, ``docs:``).

8. CONCLUSIONES Y RECOMENDACIONES

8.0 Diagrama — Vista general del sistema entregado

Figura algorítmica (diagrama de flujo). La representación gráfica completa del diagrama está disponible en la versión Markdown del informe. A continuación se resume la lógica representada.

- U[Usuario / Evaluador] --> B[Navegador :8080]
- B --> V[Vue.js web/]
- V <--> R[API REST Go]
- R <--> M[(SQL Server\nInventarioProductosDB)]
- R <--> T[Scapegoat Tree\n $\alpha = 2/3$]
- T -->|O log n amortizado| Q[Search Insert Delete]

8.1 Conclusiones de ingeniería

La implementación del Scapegoat Tree en Go cumple las invariantes teóricas del paper de Galperin y Rivest: las inserciones ordenadas disparan reconstrucciones, las eliminaciones masivas activan rebuild global, y las operaciones aleatorias coinciden con un ``map`` de referencia en 5000 iteraciones.

En la práctica, el costo de calcular ``size(u)`` recorriendo subárboles se hace tangible en secuencias adversas. La búsqueda, en cambio, es estable (~ 214 ns/op con $n=100000$, cero allocations), lo que confirma que el árbol mantiene profundidad logarítmica en ese régimen.

La separación entre SQL Server (persistencia) y Scapegoat Tree (índice en memoria) demostró ser arquitectónicamente sólida: la API REST sincroniza ambos mundos sin acoplar el paquete ``scapegoat`` a drivers de base de datos.

La simulación Vue transformó un algoritmo abstracto en un artefacto observable. Ver el nodo scapegoat pulsando en rojo mientras el subárbol se reensambla valida comprensiones que ningún diagrama estático sustituye por completo.

8.2 Limitaciones identificadas

1. Dataset acotado: 8 productos semilla; no se stress-testeó con 50,000 registros.
2. `size` sin cache: cuello de botella en detección de desbalance.
3. Sin persistencia del árbol: solo los productos viven en SQL; la forma del árbol se reconstruye al reiniciar.
4. Mutex global: la API serializa todas las operaciones; no escala horizontalmente.
5. Frontend sin build system: Vue por CDN es adecuado para demo, no para producción.

8.3 Lecciones aprendidas

Punteros en Go: los hijos `node` deben reasignarse con cuidado durante `rebuild`; un error en el reenlazado del scapegoat corrompe el árbol silenciosamente. Las pruebas contra `map` fueron indispensables.

Genéricos: `Tree[K,V]` con `less func(K,K) bool` ofrece flexibilidad; `NewOrdered` simplifica el 90% de los casos del proyecto.

Sincronización con Vue: el patrón `mutar en POST → loadTree en GET` mantiene la UI consistente, pero las animaciones requieren el snapshot previo a `loadTree` para mostrar el camino de búsqueda antes de eliminar. Ese detalle de orden temporal fue un bug sutil corregido durante desarrollo.

8.4 Recomendaciones futuras

- Almacenar `size` en cada nodo o usar análisis amortizado con contadores locales para reducir costo de detección.
- Pipeline de carga masiva con `BULK INSERT` en SQL Server y medición de tiempo de construcción del índice.
- Fuzz testing con `testing/fuzz` sobre secuencias aleatorias de operaciones.
- Migrar frontend a Vue con Vite y componentes tipados si el proyecto evoluciona beyond la demo académica.

REFERENCIAS

1. Galperin, G. F., & Rivest, R. L. (1993). Scapegoat trees. Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 165–174.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. Capítulos 12–13.
3. Go Project. (2024). Go Generics. <https://go.dev/doc/tutorial/generics>
4. Microsoft. (2024). SQL Server Documentation. <https://learn.microsoft.com/sql/>
5. Vue.js Project. (2024). Vue.js 3 Guide. <https://vuejs.org/guide/>

ANEXO A — Comandos de ejecución

Código fuente (bash)

```
# Pruebas
go test ./scapegoat/...

# Benchmarks
go test -bench=Benchmark -benchmem ./scapegoat/

# Demo SQL Server
go run ./cmd/sqlserver-demo

# Simulación web
go run ./cmd/simulation
# → http://localhost:8080
```

ANEXO B — Variables de entorno SQL Server

Código fuente (powershell)

```
$env:SQLSERVER_HOST="localhost"
$env:SQLSERVER_DATABASE="InventarioProductosDB"
$env:SQLSERVER_USER="sa"
$env:SQLSERVER_PASSWORD="TuPassword"
```

```
$env:SQLSERVER_ENCRYPT="disable"  
$env:SQLSERVER_TRUST_CERT="true"
```

Documento generado para el Trabajo Final de Algoritmos y Estructura de Datos —
Universidad ESAN — Semestre 2026-1.